



Transcript

Week 27: Computer Vision

Video 1 - Computer Vision

Ever wondered how your phone unlocks just by looking at your face? That's computer vision at work—helping machines make sense of what they see. From self-driving cars to medical imaging, this field is transforming the way technology interacts with the world. Let's explore how it all works and why it matters.

Welcome to the Computer Vision module. Before we dive into how machines see and understand images, let's look at what you'll learn by the end of this module.

First, you'll understand the core concepts behind computer vision. We'll talk about why it's such a powerful area of artificial intelligence and what makes it so challenging. You'll explore how machines interpret visual data and the foundational ideas that make this possible.

Next, you'll explore image processing techniques—these are the tools that help clean, enhance, and transform raw image data. For instance, you'll learn about image filtering, which helps remove noise, and feature extraction, which highlights important visual details like edges and shapes. We'll also look at how images are digitally represented and processed step by step.

Then, you'll identify different types of computer vision tasks. This includes image classification, which answers "what's in the picture?", and object detection, which goes a step further to find "where is it located?". You'll also learn about image segmentation, where machines label every pixel to build a detailed understanding of the scene.

After that, we'll move to hands-on coding. You'll implement basic image processing algorithms using Python and a popular library called Open C-V. You'll write real code and run it on real images, helping you see theory in action.

Finally, you'll learn how to analyse the results from your models. It's not just about building something that runs—it's about understanding what the output means, how well the model performs, and how you might improve it.

By the end of this module, you'll feel confident in your understanding of computer vision and be ready to explore more advanced topics or apply your knowledge in practical projects.

Now that you've seen what's ahead, which real-world problem would you solve first with computer vision—security, health, or automation?

Video 2 - Basic Image Processing Techniques

When you look at a photo, your brain instantly makes sense of the shapes, edges, and patterns. But for a machine, it all starts as raw pixels. How do we teach it what matters and what doesn't?

In this session on Basic Image Processing Techniques, a key part of our Computer Vision module. Before machines can understand or interpret images, we need to prepare that visual data. That's where image processing plays a vital role.

This session focuses on the core techniques that help machines clean, adjust, and improve



images before any further analysis takes place. These steps come before classification, detection, or recognition—and they lay the groundwork for all of it.

We'll start by learning how to filter images, which helps reduce noise and makes important features stand out more clearly. Then, we'll move on to edge detection, where the goal is to identify outlines, shapes, and boundaries within an image—crucial for spotting objects.

Next, we'll explore image enhancement methods, which allow us to adjust contrast, sharpen details, and highlight key regions. These enhancements make it easier for both humans and machines to interpret what's in the image.

By the end of this session, you'll understand how these techniques work and why they're essential in every computer vision workflow. You'll also see how these methods connect directly to real-world tasks—like preparing X-rays for diagnosis, improving satellite images, or powering visual search engines.

Let's get started by looking at how raw images are transformed into usable visual data.

Before we jump into the techniques, let's quickly recap where we are in the bigger picture.

Computer Vision is all about teaching machines to understand visual data, just like humans do when we look around. But unlike us, machines don't naturally recognise shapes or scenes. Everything starts as raw pixel values—just numbers.

Through computer vision, we give machines the ability to identify what's in an image, where it is, and how to describe it.

So, what exactly is image processing?

At its core, it's the process of using computers to work with digital images—improving how they look, pulling out useful information, or getting them ready for deeper analysis. Think of it as preparing the image before asking the machine to interpret what's in it.

The input is usually a raw digital image, made up of pixel values. This could be a photograph, a scanned document, or a frame from a video. The output can take two forms. It might be another image that's cleaner or clearer, or it could be a set of values—features or measurements—extracted from the image for further processing.

For example, you might apply image processing to sharpen a blurry X-ray, remove noise from a satellite photo, or extract edges and patterns from handwritten text. All of this helps the machine focus on what matters most in the image.

In computer vision, this step lays the foundation for everything that follows—from object recognition to scene understanding.

Before a machine can understand what's in an image, it first needs to process that image. That's where image processing comes in—it helps improve image quality, extract important details, and prepare data for deeper analysis.

Why is this so important for computer vision? Because real-world images are rarely clean. They can be noisy, inconsistent, or captured under different lighting conditions. Image processing helps machines make sense of that.

One key step is noise reduction, where we remove visual artefacts—like blur or static—that could interfere with accuracy. Then comes feature extraction, where we highlight useful patterns like edges, textures, or corners, which are essential for recognising objects.

We also use normalisation to make sure images are consistent—by adjusting brightness, contrast, or size—so the input stays reliable. After that, preprocessing gets images ready for AI models, by cleaning, resizing, or formatting the data for smooth analysis.



All these steps lead to improved accuracy, helping models detect and classify images more effectively. That's why image processing plays a foundational role in the computer vision pipeline—it sets the stage for every task that follows.

And it's not just theory. In real-world applications, image processing is used to clean up CCTV footage, sharpen medical scans, and align different image types so they can be compared and analysed properly.

In this session, we'll look at how these techniques work, why they matter, and how to apply them in practical scenarios.

Before we can manipulate or analyse an image, we first need to understand how it's represented inside a computer.

Digital images are made up of tiny units called pixels, short for "picture elements". You can imagine an image as a two-dimensional grid where each square in the grid is one pixel. Each pixel stores a value that tells the computer something about that spot—usually its brightness or colour.

For a grayscale image, each pixel holds just one value. This is a number between zero and two hundred and fifty-five, where zero means black, two hundred and fifty-five means white, and values in between represent shades of grey.

For colour images, things are a bit more complex. Most colour images use the RGB model, which stands for Red, Green, and Blue. Each pixel in this case stores three values—one for each colour channel. Again, each channel ranges from zero to two hundred and fifty-five. By combining different intensities of red, green, and blue, we can represent millions of colours in a single image.

Finally, we come to resolution, which refers to the total number of pixels in an image. This is usually expressed as width by height—for example, nineteen twenty by ten eighty, or 1920×1080 pixels. The higher the resolution, the more detail we can capture.

Understanding this pixel-based structure is essential because nearly all image processing techniques involve working directly with these pixel values.

Let's now move into our first core technique in image processing—image filtering.

The main goal of filtering is to modify the value of each pixel based on the values of its neighbouring pixels. This is especially useful for reducing noise or smoothing an image to make it clearer.

At the heart of most filtering operations is a mathematical process called convolution. This operation applies a small grid of numbers, called a kernel, to the image. The kernel is also known as a filter or mask.

A kernel is typically a 3 by 3 or 5 by 5 matrix. It contains weight values that we slide over the image, one pixel at a time. At each step, the pixel's value is updated based on the weighted combination of its surrounding pixels.

For example, a smoothing filter averages the surrounding pixels to blur the image gently and reduce sharp noise. On the other hand, an edge detection filter highlights differences between neighbouring pixels, helping us identify outlines or boundaries in the image.

Understanding this concept is essential because filtering is the foundation for many practical tasks in computer vision, from cleaning up camera feeds to preparing medical scans.

Let's break down how image filtering actually works using a method called convolution.



First, we place a small matrix called a kernel over a specific part of the image.

The kernel is usually a 3 by 3 or 5 by 5 grid of numbers, also called weights.

We centre the kernel on one pixel at a time.

Next, we multiply each value in the kernel by the pixel value directly underneath it in the image. For example, if the top-left kernel weight is two, and the corresponding image pixel is one hundred, we multiply two by one hundred.

After multiplying all the matching pairs, we add up the results. This final sum becomes the new value for the centre pixel. Then we slide the kernel to the next position and repeat the process.

We do this for every pixel in the image. The result is a new version of the image, where each pixel has been adjusted based on its neighbours.

This process is the foundation for many tasks—like smoothing out noise, sharpening edges, or highlighting patterns—and it's used throughout computer vision.

When our goal is to reduce sharp transitions, blur fine details, or suppress noise, we use blurring and smoothing filters. Each type of filter works a little differently, with its own strengths and trade-offs.

Let's start with the average filter, also called the mean filter. This filter takes the average of a pixel and its neighbours inside the kernel. The result replaces the central pixel. It's simple and quite effective for reducing random noise—like salt-and-pepper noise, which appears as scattered black and white dots. But one drawback is that it can blur edges and important details.

Next is the Gaussian filter, which improves on the average filter. Instead of giving equal weight to every pixel, the Gaussian filter gives more importance to pixels closer to the centre. This is done using a bell-shaped weight pattern called a Gaussian distribution. The result is a more natural-looking blur that softens noise while preserving more of the image's structure.

Finally, we have the median filter, which is especially good at removing salt-and-pepper noise. Unlike the other two, this filter replaces the central pixel with the median value of its neighbours—not the average. That means it's less affected by extreme pixel values. It also does a better job of keeping sharp edges intact.

Understanding when and why to use each of these filters is essential for building effective computer vision systems.

Now that we've looked at smoothing, let's move on to another key technique—edge detection.

The main purpose of edge detection is to find points in an image where the intensity of pixels changes rapidly. These changes often mark the boundaries of objects or important visual features.

So, what do we mean by 'edges'? An edge is where one part of the image ends and another begins. This could be the line between two objects, a sudden change in texture or surface, or even a shift in lighting. Edges often carry the most important information in an image because they show where something changes.

To find these edges, we use filters that highlight intensity gradients—in other words, the difference between pixel values. Instead of smoothing things out like a blur filter, edge detection does the opposite. It sharpens areas of contrast so that strong edges show up clearly—usually as bright lines on a dark background.



Edge detection is essential in many computer vision tasks. It helps with object recognition, where finding the shape or boundary of an object is the first step. It also supports segmentation, where an image is divided into useful parts, and feature extraction, where we gather information about the object's shape or structure.

By understanding where and how intensity changes, we allow machines to see the structure of an image—just like our eyes do when we recognise shapes and outlines.

Let's now look at one of the most widely used edge detection methods—the Sobel operator.

The Sobel operator is known for being simple and fast. It uses two small matrices, or kernels, each sized 3 by 3. One kernel is designed to detect edges that run vertically, and the other detects horizontal edges. These kernels are applied to the image through a process called convolution—spelled C-O-N-V-O-L-U-T-I-O-N.

Once both kernels have been applied, the results are combined to calculate two things: the magnitude of the gradient, which tells us how strong the edge is, and the direction of the gradient, which tells us where the edge is pointing.

This makes the Sobel operator very useful when we want to identify both the shape and orientation of objects in an image. However, it's important to note that the Sobel filter can also pick up unwanted variations caused by noise. That means small changes in pixel values—such as random specks—might be incorrectly flagged as edges.

Despite that limitation, the Sobel operator remains a popular starting point for edge detection in many vision systems because of its efficiency and ease of use.

Now that we've explored the Sobel operator, let's look at two other widely used edge detection methods—the Prewitt operator and the Canny edge detector.

First, the Prewitt operator is very similar to Sobel. It also uses two three-by-three kernels—one for detecting vertical edges and the other for horizontal edges. However, it uses simpler numerical values for its kernels. That makes it slightly faster but generally more sensitive to noise. Like Sobel, it works by highlighting where the image changes quickly.

Next, we come to the Canny edge detector, which is often considered the gold standard. It's a multi-stage algorithm that solves many of the limitations of simpler filters. Here's how it works:

It begins with Gaussian smoothing to reduce noise. Then, it calculates the gradient magnitude and direction—just like the Sobel filter. After that, Canny performs non-maximum suppression—this step keeps only the sharpest edges by removing any weak or blurred responses nearby.

Finally, it uses something called hysteresis thresholding to connect strong edges and eliminate stray ones. The result is a clean, thin, and continuous set of edges—ideal for tasks like object recognition, image segmentation, or shape analysis.

Next is Image Enhancement. This technique focuses on improving the visual quality of an image—either to make it more appealing to the human eye or more suitable for further analysis by computer vision systems.

It's important to understand the difference between enhancement and restoration. Restoration tries to recover an original, undistorted image based on a model of how it was degraded. In contrast, enhancement is more subjective—it's about making the image look "better" for a specific purpose or audience.

Some typical enhancement goals include improving contrast, adjusting brightness,



sharpening fine details, and correcting colors so that the image appears more natural or vibrant.

Let's explore some practical techniques to enhance image quality.

First, contrast stretching, also called normalization, is a simple method that expands the range of pixel intensities. For example, if an image only uses pixel values between 50 and 200, this method remaps those values to the full range from zero to two fifty-five. This makes the image details more visible and easier to analyse.

Next, we have histogram equalisation, which redistributes pixel intensities to produce a more balanced histogram. It's especially helpful for improving contrast in images with very bright or dark areas, giving a more even spread of intensities.

Finally, sharpening filters help emphasise edges and fine details. Unlike blurring, these filters use special kernels to highlight the differences between nearby pixels. Examples include the Laplacian filter and a method called Unsharp Masking. But keep in mind, over-sharpening can make noise more visible and introduce unwanted effects.

Let's now see how we can actually implement these techniques in practice.

For that, we'll use a powerful and widely adopted library called OpenCV,

which stands for Open Source Computer Vision Library.

It supports Python, C plus plus, and Java, and makes it easy to perform a wide range of image processing tasks.

To load an image, we use the function `c-v-2-dot-i-m-read`.

To display the image, we use `c-v-2-dot-i-m-show`.

To convert between color spaces—for example, from B-G-R to grayscale—we use `c-v-2-dot-c-v-t-color`.

For filtering, we have several tools.

`c-v-2-dot-filter-2-D` is used when we want to apply custom two-dimensional convolution using our own kernel.

For standard blurring, we have `c-v-2-dot-blur`,

`c-v-2-dot-gaussian-blur`,

and `c-v-2-dot-median-blur`.

For edge detection, we rely on two key functions:

`c-v-2-dot-sobel` and `c-v-2-dot-canny`.

And for enhancing contrast through histogram equalisation, we use `c-v-2-dot-equalize-hist`.

In our upcoming practical session, we'll use these functions to apply real image transformations,

showing how just a few lines of Python code can bring powerful results in computer vision.

How could you apply these techniques to improve facial recognition, medical scans, or autonomous driving? Explore the possibilities in your next project.



Video 3 - Image Scaling

What happens when you resize an image, but the quality falls apart? In computer vision, scaling images the wrong way can distort what machines learn. So how do we preserve detail while changing size? Let's explore the smart techniques behind accurate image scaling.

Image scaling means resizing an image, but it's not just about changing the height and width. It's also about preserving quality while transforming the image's size.

To scale an image properly, we must determine the new pixel values for the resized image. This brings us to interpolation—the process of estimating values between known pixels. Interpolation plays a crucial role in maintaining image clarity, especially when resizing for computer vision applications like object detection, facial recognition, or medical imaging.

In this session, we'll cover several key areas. First, we'll define what image scaling is and understand why it's so important in modern computer vision workflows. Then, we'll explore how interpolation helps us fill in missing pixel information when an image is resized.

We'll focus on three of the most common interpolation techniques: Nearest Neighbour, Bilinear, and Bicubic. You'll learn how each method works, their pros and cons, and when to use them. For example, Nearest Neighbour is simple and fast, but may lead to blocky results.

Bilinear offers smoother results by averaging nearby pixels, while Bicubic uses more complex calculations for even finer detail, though it's slower.

Finally, we'll look at real-world applications where image scaling is crucial, and we'll introduce how to implement these techniques using the Open-C-V library in Python.

Let's begin by understanding what image scaling means. In simple terms, it's the process of resizing a digital image. This could involve enlarging, or zooming in, and shrinking, or zooming out.

But scaling isn't just about changing the size. It's a vital step in many computer vision tasks. Let me explain why.

First, we have normalisation. Many deep learning models, like convolutional neural networks, need input images to be the same size—such as two-two-four by two-two-four pixels. Image scaling ensures all images meet this requirement.

Next is memory efficiency. Resizing images helps reduce file size. This allows faster processing and saves memory—especially useful when dealing with large datasets.

Then we have display optimisation. Images must be scaled to fit different screens, layouts, or web platforms correctly—without distortion or long loading times.

Feature matching is another reason. In tasks like object tracking or panorama stitching, images are often resized to create what we call image pyramids. This allows models to detect patterns at multiple scales.

And finally, in deep learning, data augmentation uses image scaling to create variations of training images. This makes models more robust and helps prevent overfitting.

When we resize an image, we're not just stretching or shrinking it—we're also creating or combining pixel values. That's where the challenge lies. Let's start with downscaling. When we shrink an image, several original pixels might compress into a single one. In that case, we typically use the average or one representative value to fill the new pixel. This is



relatively simple.

But the real complexity begins with upscaling. When we enlarge an image, a single original pixel now has to stretch to cover more space. That creates empty pixel locations—places where we need to fill in colour or intensity information that wasn't there before. But how do we guess those new values accurately?

This is where interpolation becomes crucial. Interpolation is the mathematical technique used to estimate the missing values by using surrounding known pixels. In essence, it helps us "fill in the blanks" intelligently to ensure the resized image still looks natural, detailed, and usable for computer vision tasks.

Let's delve deeper into interpolation. In mathematics, interpolation is a method of constructing new data points that lie between a set of known values. In the context of image scaling, particularly during upscaling, interpolation helps estimate the intensity or colour value of a target pixel in the resized image. Since these new pixel positions didn't exist in the original image, we use the values of surrounding, known pixels to make an educated guess.

This is critical for maintaining image quality. The way we select and combine the surrounding pixels defines which interpolation method we're using. Some methods take the nearest known value. Others calculate averages or fit curves to produce smoother results.

There are trade-offs. Simpler methods compute quickly and are ideal for real-time applications, but may look blocky or rough. More complex methods produce cleaner, more accurate images—but they're slower and demand more processing power.

So whenever we scale an image, especially for applications like facial recognition or medical imaging, we must balance three key factors—speed, image quality, and computational cost. Choosing the right interpolation technique depends on the use case.

Our first interpolation technique is called Nearest Neighbour Interpolation, and as the name suggests, it's the simplest and fastest method available.

Here's how it works: for every new pixel location in a resized image, the algorithm finds the closest pixel in the original image and assigns its exact value to the new pixel. There's no averaging or blending involved—it just picks the nearest neighbour and copies its value.

This method is extremely fast and requires very little computational power, which is why it's used in applications where speed is a priority and image quality is less important.

But this simplicity comes at a cost. Nearest Neighbour Interpolation often introduces visual artefacts. For example, you might see "jaggies"—those staircase-like distortions that appear along curved or diagonal edges. This happens because no smoothing is applied between pixel transitions.

Another issue is poor quality during significant upscaling. Since it just replicates the original pixels, the image may look blocky or pixelated, especially when zoomed in. This makes it unsuitable for producing high-resolution images where clarity is essential.

In summary, Nearest Neighbour is fast and simple, but comes with visible trade-offs in visual quality—especially during upscaling or when smooth transitions are needed.

Let's now look at a more refined method: Bilinear Interpolation. Compared to Nearest Neighbour, this approach is a step up in both complexity and quality. Instead of just one nearest pixel, it considers the four closest original pixels surrounding the new pixel's position.

Here's how it works: for each new pixel, the algorithm first locates its fractional position in the original image. It then identifies the four surrounding original pixels and calculates a



weighted average of their values. The closer an original pixel is to the new pixel, the more influence its value has. In other words, nearer pixels get higher weights.

This weighted averaging results in a much smoother output, significantly reducing visual issues like aliasing—those jagged edges—and blockiness seen in Nearest Neighbour. It also maintains a moderate processing speed, making it a practical choice when both quality and efficiency are needed.

But it's not perfect. Bilinear Interpolation can blur fine details, especially along sharp edges, because it blends pixel values together. And while better than Nearest Neighbour, it still isn't suitable for very large scaling, where high detail is required.

Overall, it offers a good middle ground between performance and quality—ideal for many general-purpose image scaling tasks.

Now we move to the most advanced method—Bicubic Interpolation. This approach is more complex than Nearest Neighbour and Bilinear. Instead of using just one or four surrounding pixels, Bicubic considers the sixteen nearest pixels in a four-by-four grid around the target pixel.

For each new pixel, the method applies a cubic polynomial function to calculate a weighted average of these sixteen values. But it doesn't stop there—it also considers the gradients, or how the values are changing between pixels, to estimate the curve of the image more accurately. This allows Bicubic to predict values more smoothly and with greater precision.

The result? Exceptionally smooth transitions, sharper edges, and much higher quality than the other two methods—especially when you're enlarging an image. Bicubic also does a better job at reducing aliasing, which means fewer jagged or blocky artefacts.

However, there is a trade-off. Bicubic Interpolation is the slowest and most computationally demanding of the three techniques. It takes longer to process and isn't ideal for real-time tasks or large batches of images. And in some cases, it can create ringing artefacts—these are faint halo-like patterns that sometimes appear around sharp lines or edges.

Still, when quality is the top priority, especially for upscaling, Bicubic is often the best choice.

With all three primary interpolation methods in mind—Nearest Neighbour, Bilinear, and Bicubic—the obvious question is: which one should you choose? The answer is, there's no single best method that works for every situation. The right choice depends entirely on your specific use case and what matters most for your application.

There are three key factors you need to think about. The first is speed versus quality. Is your task time-sensitive? For example, do you need to resize images quickly in real time, or can you afford a short delay if it means better visual quality? A faster method might be more useful in time-critical systems, while a slower one might be worth it if appearance is the priority.

The second factor is the scaling level—how much are you enlarging or shrinking the image? For small changes in size, there may not be a major visible difference between Bilinear and Bicubic. But when the scaling factor is large—say you're doubling the size of an image—Bicubic will usually produce much better results.

The third factor is the image content itself. Does your image have fine details, sharp lines, or areas with high contrast? Or is it mostly made up of smooth gradients and textures? The type of content will influence which artefacts are more noticeable and which method handles them best.

Let's look at some general recommendations. Nearest Neighbour is the fastest method and



is ideal when speed is the top priority or when you're working with very small images. It also works in cases where the blocky, pixelated look is acceptable or even desired, like with retro-style visuals.

Bilinear Interpolation provides a good middle ground—it offers a better visual outcome than Nearest Neighbour while still being fast enough for most general uses. That makes it a practical choice when you're dealing with moderate upscaling and general-purpose tasks.

Bicubic Interpolation is the most advanced of the three. It delivers the highest quality, especially for upscaling, and helps preserve fine details and smooth edges. However, it's also the most computationally intensive, so it's best suited to offline processing, where performance is less of a concern and image quality is paramount.

Finally, if you're downscaling—reducing the image size—there's another method worth mentioning: Area Interpolation. It's often the default in libraries like OpenCV. Instead of focusing on a few nearby pixels, it resamples the entire area being shrunk and averages the pixel values across that region. This results in smoother images and helps avoid aliasing artefacts. That makes Area Interpolation particularly effective when reducing image size while preserving clarity.

Let's now see how to scale images in Python using the Open-C-V library. The main function we use is `c-v-two dot resize`. This function is part of OpenCV and makes image resizing straightforward and flexible.

Here's how it works. First, we provide the input image, referred to as `s-r-c`. Next, we define the output size using a parameter called `d-size`. This is written as a pair of values—width and height. If you already know the exact dimensions you want, you simply pass them into `d-size` as a tuple.

Alternatively, instead of giving a fixed size, you can scale the image by a certain factor using `f-x` and `f-y`. These are the scaling values for the horizontal and vertical directions. For example, setting `f-x` equals zero point five will reduce the width of the image by half. When using `f-x` and `f-y`, the `d-size` value is usually set to zero.

Now comes the most important part—the interpolation flag. This is where you choose the interpolation method. For the fastest option, you can use `c-v-two dot INTER underscore NEAREST`, which uses Nearest Neighbour interpolation. For general resizing tasks with better quality, use `c-v-two dot INTER underscore LINEAR`, which applies Bilinear interpolation. If you want even smoother results and sharper edges, go with `c-v-two dot INTER underscore CUBIC`, which uses Bicubic interpolation.

Finally, if you're shrinking an image rather than enlarging it, the best choice is usually `c-v-two dot INTER underscore AREA`. This method averages pixel values in the shrinking region and gives smoother, cleaner results.

So, to summarise: using `c-v-two dot resize`, you can choose whether to resize by fixed dimensions or by scale, and you can control how smooth the result looks using the interpolation method. This makes it a powerful and flexible tool for image processing in Python.

Let's walk through a practical Python example using the OpenCV library to resize images with different interpolation methods.

We begin by importing `c-v-two`, then load an image using `c-v-two dot i-m-read`, which takes the file name of the image as a string. In this case, it's `"my underscore image dot j-p-g"`.

Next, we downscale the image to half its size. This is done using the `f-x` and `f-y` parameters, both set to zero point five. These represent the horizontal and vertical scaling factors. Because we haven't specified a fixed output size using `d-size`, OpenCV calculates



the new dimensions using these scale factors. The interpolation method here is set to c-v-two dot INTER underscore LINEAR, which is Bilinear interpolation. This method is the default in OpenCV and offers a good balance between performance and quality, making it suitable for general resizing.

After that, we upscale the image to a specific size—six hundred and forty by four hundred and eighty pixels. Here, we pass the output size directly using d-size and apply c-v-two dot INTER underscore CUBIC, which uses Bicubic interpolation. This method produces very smooth results with sharper edges and is recommended when you want to preserve image quality while enlarging.

In the third example, we resize the image using Nearest Neighbour interpolation. This is done by passing c-v-two dot INTER underscore NEAREST to the interpolation parameter. It is the fastest method and uses the closest pixel value, but it can make the result look blocky or jagged—especially around curves or sharp edges.

Finally, the resized images and the original image are displayed using c-v-two dot i-m-show, and the script waits for a key press using c-v-two dot wait-key, before closing all windows with c-v-two dot destroy-all-windows.

So, in just a few lines of code, we've demonstrated three powerful resizing methods using OpenCV: Nearest Neighbour, Bilinear, and Bicubic—each with its own trade-offs and ideal use cases.

Which interpolation method would you choose for a mobile app that streams video in real time? Why?

Video 4 - Object Detection Algorithms

What if your computer could not only see an image—but also tell you exactly what's in it and where it is?

Now that we've explored the fundamentals of image processing and scaling, it's time to move into one of the most exciting areas of computer vision—object detection. This is where machines go beyond simply recognising that an image exists—they start to understand what's in it, and where those objects are located.

Object detection allows a system not just to see, but to identify and locate specific elements inside an image—like a pedestrian in a street scene, or a car on the road. It's a crucial step for real-world applications such as autonomous driving, medical imaging, surveillance, and even augmented reality.

In this session, we'll explore the core ideas behind object detection—a powerful concept in computer vision that goes beyond simply recognising what's in an image. Unlike image classification, which only tells you what category an image belongs to, object detection allows a system to both identify and locate multiple objects within the same image.

We'll start by understanding what makes object detection more complex, especially the challenge of classifying and localising objects at the same time. You'll see how the field has evolved—from traditional techniques to deep learning-based models that now lead the way.

We'll focus on two major types of object detection models. First, we'll look at two-stage detectors, like Faster R-C-N-N, which work by first proposing possible regions of interest and then classifying what's inside them.

Then we'll explore one-stage detectors, such as Y-O-L-O, short for You Only Look Once, and S-S-D, or Single Shot Detector. These models perform detection in just one step,



which makes them much faster and more suitable for real-time applications.

Finally, we'll compare these models side by side—examining their trade-offs in terms of speed, accuracy, and use case—and we'll wrap up by looking at how object detection is applied in the real world, from self-driving cars to security systems and retail automation.

Let's start by clearly defining Object Detection. It's a core task in computer vision that goes beyond simply recognising what's in an image. Object detection not only identifies what objects are present, but also where they are, by drawing bounding boxes around each one.

These objects are usually things like people, animals, cars, or traffic lights—what we call semantic objects. To locate them, detection algorithms output rectangular coordinates that mark the top-left corner and define the width and height of the box.

To understand the value of object detection, it helps to compare it with image classification. Classification tells you the most likely label for the whole image—for example, "This image contains a cat." But detection gives a more complete picture. It answers: "What are all the objects in this image, and where are they?" So, instead of just "cat," the output might be "Cat at these coordinates, person at these coordinates, and car over here."

For each object detected, the model usually provides three pieces of information. First, the class label—such as 'person' or 'bicycle'. Second, a confidence score, which tells us how sure the model is about its prediction. And third, the bounding box coordinates, usually given as x and y for the top-left corner, along with the width and height of the box.

In short, object detection helps machines not only see—but understand and locate—multiple objects in an image or video.

Object detection is one of the most challenging problems in computer vision because of how complex and varied real-world scenes can be. It's not just about finding one clearly visible object—it's about identifying many, often overlapping, often distorted, and all in a single image.

Let's break down the difficulties. First, there's scale. Objects can appear in many sizes—a car up close takes up most of the frame, but the same car in the distance might appear tiny. Next is viewpoint—we might see an object from the front, from the side, or even from above. Then there's occlusion. That means some objects may be partially hidden—like a person behind a tree or a bicycle blocked by a car.

Deformation is another challenge. Unlike rigid objects, people and animals can bend, twist, or move in unpredictable ways. This makes it harder to detect them accurately. In real scenes, you also get clutter—multiple objects packed into the same frame, often overlapping or blending into the background. And don't forget lighting changes—from direct sunlight to shadows or low-light conditions, these all affect how objects appear to a computer vision system.

But the biggest challenge is that the algorithm must perform two tasks at once—it needs to localise each object by drawing a bounding box around it, and also classify what that object is. And it has to do this for every object in the scene. That's what makes object detection both computationally intensive and intellectually fascinating.

Object detection has gone through a major transformation—especially with the rise of deep learning. In the early years, detection relied on hand-crafted features. That means developers manually created rules or filters to detect specific patterns in images. For example, they used Haar cascades for face detection or H-O-G—Histogram of Oriented Gradients—for identifying pedestrians.

These methods often used a sliding window approach—where the system scanned every



possible region of the image at different sizes. While this sometimes worked, it was slow, inefficient, and produced many false positives. One of the more successful early models was Viola-Jones, which could detect faces in real time.

But everything changed around 2012, with the start of the deep learning revolution. Convolutional Neural Networks, or C-N-Ns, made it possible to automatically learn useful features from data. This removed the need for manually designing detectors.

The first major breakthrough came with R-C-N-N, or Region-based CNN. It applied C-N-Ns to object detection—but it was still slow. Later versions like Fast R-C-N-N and Faster R-C-N-N improved speed and accuracy by proposing regions and classifying them in one pipeline. These are known as two-stage detectors.

Then came YOLO, short for You Only Look Once, and S-S-D, or Single Shot Detector. These models detect objects in a single pass through the network, making them much faster. They're often used for real-time applications where speed matters more than perfect accuracy.

This shift from hand-crafted rules to learned features has made object detection faster, more scalable, and far more accurate.

Let's explore Faster R-C-N-N, one of the first successful deep learning models for object detection. It's known as a two-stage detector, which means it processes the image in two main steps.

Stage one is called the Region Proposal Network, or R-P-N. This is a small neural network that scans the image and suggests regions that might contain objects. These suggested areas are called regions of interest, or RoIs. Unlike the older sliding window method, R-P-N makes this step much more efficient by only selecting promising locations.

Stage two is the Faster R-C-N-N head. It takes each proposed region and does two things. First, it performs classification—figuring out what object is inside that region, like a 'dog', 'car', or 'traffic light'. Second, it does bounding box regression, which fine-tunes the coordinates of the box so it tightly fits the object.

The key strength of Faster R-C-N-N is its high accuracy and precise localisation, especially for detecting smaller objects in a scene. But it has a drawback—it's slower than the one-stage models we'll look at next, so it's less suitable for real-time applications.

Now let's look at YOLO, which stands for You Only Look Once. YOLO is a one-stage detector, which means it predicts everything in a single forward pass through the network—bounding boxes, object labels, and confidence scores—all at once.

Here's how it works: The image is first split into a grid, like seven by seven. Each grid cell is responsible for predicting a fixed number of bounding boxes. For each box, YOLO outputs a confidence score—which tells us how likely the box contains an object—and a class label, like 'cat' or 'car'.

After that, Non-Maximum Suppression, or N-M-S, removes overlapping boxes and keeps only the most confident ones.

YOLO's big advantage is speed. Because it sees the entire image at once, it's capable of detecting objects in real time—even in video streams.

But the trade-off is accuracy. YOLO can struggle with small objects in crowded scenes, or with objects that have unusual shapes or aspect ratios.

Our second major one-stage detector is SSD, which stands for Single Shot Detector. SSD is designed to strike a balance between the speed advantages of YOLO and the localisation accuracy offered by two-stage detectors like Faster R-C-N-N. What sets SSD apart is that it



performs both object localisation and classification in a single forward pass through the network, without breaking the process into stages.

SSD uses a single convolutional neural network, but instead of making predictions from just the final layer, it predicts from multiple layers of the network. These layers operate at different resolutions—deeper layers with larger receptive fields are better for detecting big objects, while earlier layers with finer details are better at picking up smaller ones. This approach is known as multi-scale prediction.

In addition to this, SSD uses predefined default boxes, also called anchor boxes, that come in different aspect ratios and sizes. These boxes are positioned across each location in the feature maps. For each box, SSD predicts the object's class, adjusts the box's coordinates to better fit the object, and provides a confidence score that tells us how likely it is that an object is present.

Once all the predictions are made, SSD applies a technique called Non-Maximum Suppression. This method helps filter out overlapping boxes by keeping only the most confident and accurate ones, reducing redundancy in the final results.

SSD's biggest strengths lie in its speed and accuracy. It's faster than Faster R-CNN and generally more accurate than the original YOLO model. It also performs particularly well when detecting objects of different sizes in the same image, thanks to its multi-scale design.

However, SSD does come with a few trade-offs. It can still struggle to detect very small objects when compared to two-stage methods. And while it is fast, it may not achieve the extremely high frame rates that newer versions of YOLO can reach.

Let's step back and compare the three major object detection models we've studied so far—Faster R-CNN, YOLO, and SSD. Each of them takes a different approach, and the best choice depends on your use case.

Faster R-CNN is a two-stage detector that prioritises accuracy and precision. It performs extremely well for tasks that need high localisation accuracy, such as medical or industrial inspection. However, because it runs in two stages—region proposal followed by classification and box regression—it's slower, usually running at around 5 to 15 frames per second. That makes it better suited for offline or non-real-time scenarios. Its strength lies in detecting small and detailed objects with high reliability.

In contrast, YOLO, which stands for "You Only Look Once," takes a one-stage approach. It processes the entire image in a single forward pass, which makes it incredibly fast—ranging from 45 to over 150 frames per second depending on the version. This speed makes it ideal for real-time tasks like video surveillance or autonomous driving. Earlier versions had moderate accuracy, but newer YOLO versions have closed that gap. The model is simpler and end-to-end, though it still struggles with detecting small objects in cluttered scenes or with odd aspect ratios.

SSD, or Single Shot Detector, offers a middle ground. It's also a one-stage detector, but it improves accuracy by using multiple scales and default boxes across different feature layers. This allows it to better detect objects of varying sizes, outperforming YOLO on smaller objects while still being faster than Faster R-CNN. SSD runs at about 20 to 60 frames per second, depending on configuration. It's moderately complex and often used for general-purpose detection in real-time systems.

It's worth noting that these frame rate numbers are approximate and can vary widely based on implementation, hardware, and image resolution. Ultimately, choosing the right model comes down to what matters more for your task—speed, accuracy, or the ability to detect small and detailed objects.



The power to locate and identify objects with precision has transformed industries across the board. Object detection is no longer just a research topic—it's a core technology that shapes how modern systems see and respond to the world.

Let's look at some real-world examples.

In autonomous vehicles, object detection is absolutely essential. It helps self-driving cars identify pedestrians, other vehicles, cyclists, and traffic signs in real time—crucial for safe navigation.

In security and surveillance, object detection drives smart systems that can flag unusual behaviour, detect intruders, spot suspicious packages, or monitor crowd movement instantly.

In retail analytics, it helps businesses understand how customers move through stores, which products they interact with, how well shelves are stocked, and even powers automated checkout.

In medical imaging, object detection algorithms assist doctors by identifying anomalies, locating tumours in X-rays or MRIs, and even segmenting organs. This can significantly speed up diagnosis.

In robotics, it allows robots to perceive and interact with their environment—whether that's identifying parts for manipulation, navigating complex spaces, or even recognising human gestures.

In sports analytics, it's used to track players, detect the ball, and analyse team formations and individual performances. This provides valuable data for coaching and commentary.

In agriculture, object detection supports precision farming. It monitors crop health, checks for disease or ripeness, and even counts livestock to help farmers manage resources more effectively.

And in quality control, it's used in manufacturing lines to spot defects in real time, helping companies ensure consistent quality before products leave the factory.

These examples show just how widespread and impactful object detection has become in shaping our technological landscape.

How many of these real-world problems can you imagine solving with object detection?

Video 5 - Image Segmentation Algorithms

What if machines could not only detect objects—but understand their exact shape, pixel by pixel?

We've seen how computer vision allows machines to understand what they see—from recognising shapes and patterns to detecting and locating entire objects using bounding boxes. But what if we want the machine to go deeper, to understand the exact shape and boundary of each object?

That's where image segmentation comes in.

In this next stage, we move beyond bounding boxes to achieve pixel-level understanding. Instead of just identifying what and where an object is, segmentation techniques help us understand its exact outline—pixel by pixel.

We'll explore two major types of segmentation: semantic segmentation, which labels every



pixel that belongs to a particular category, and instance segmentation, which goes a step further and identifies individual objects—even if they belong to the same category.

This level of detail allows for highly precise visual understanding and is key in fields like medical imaging, autonomous vehicles, and robotics, where accuracy at the pixel level can make all the difference.

Let's take a closer look at how these segmentation algorithms work—and how they push the boundaries of machine vision even further.

In this session, we'll explore how image segmentation takes us deeper than object detection by focusing on pixel-level understanding. We'll begin with the basics—what image segmentation means and how it helps us label every part of an image with greater precision than bounding boxes allow.

There are two main types you'll need to know: semantic segmentation, where each pixel is assigned to a class like road, car, or background—and instance segmentation, which takes it further by identifying individual objects, even if they're from the same class.

We'll also introduce two important algorithms. First, U-Net, often used for semantic segmentation in medical imaging. And second, Mask R-CNN, which is widely used for instance segmentation in more complex scenes.

By the end of this session, you'll be able to tell the difference between these methods, understand when to use each, and appreciate how image segmentation is applied in real-world fields like healthcare, robotics, and beyond.

Let's start by breaking down what image segmentation means. It's the process of dividing a digital image into multiple segments, or sets of pixels, often called "superpixels." The main goal is to simplify or change how an image is represented—making it easier and more meaningful for a machine to understand. In essence, we assign a unique label or category to every single pixel in the image.

To understand why this matters, let's compare it with what we've already learned. In image classification, the model looks at the whole image and gives it one label—like "This is a cat." In object detection, the model goes a step further by drawing a bounding box around that cat and saying, "There's a cat at these coordinates."

But image segmentation digs even deeper. It asks, "Which specific pixels belong to the cat?" It outlines the entire shape of the object—pixel by pixel—separating it from the background or other objects. This kind of detailed segmentation is crucial in tasks where precision really matters, like in medical imaging or robotics.

The output is typically a segmented image or a mask, where every pixel is color-coded or labelled according to its category or object. This pixel-level accuracy is what makes image segmentation such a powerful tool in computer vision.

Let's now look at the two main types of image segmentation—semantic segmentation and instance segmentation—each serving a distinct purpose.

We'll start with semantic segmentation. The goal here is to classify every single pixel in an image into a predefined category or class. Imagine a scene with cars, roads, people, and sky. The algorithm assigns a class label like "car," "road," "sky," or "person" to each pixel in that image. But it doesn't tell us which specific car or person—just that a pixel belongs to the car category.

This means semantic segmentation does not distinguish between multiple objects of the same type. If there are three cars in the image, they all get the same "car" label. It's like colouring in a picture where each area is filled based on the general category it belongs to, not individual identity.



This method gives us a high-level understanding of the scene and is commonly used in autonomous vehicles, satellite imagery, and medical scans.

Now let's move on to the second type: instance segmentation.

This technique goes a step beyond semantic segmentation. Its goal is to identify and label each distinct object instance in an image. So, instead of just tagging every pixel as "car" or "road," it also assigns a unique ID to each object—even if they're the same class.

For example, if your image has three cars, instance segmentation will label them separately as "Car 1," "Car 2," and "Car 3," each with its own mask. This helps machines tell them apart, even though they all belong to the same category.

You can think of this like drawing a separate outline around each object—like tracing each car one by one—rather than coloring them all in the same shade.

Let's look at a widely used architecture for semantic segmentation—U-Net.

The core challenge in semantic segmentation is to identify what type of object is in an image and also determine which exact pixels belong to it. That means we need both a global view for object context and local detail for precise boundaries.

U-Net is a convolutional neural network architecture originally designed for biomedical image segmentation, but its flexible structure has made it popular across many applications.

It's known for its distinctive "U" shape.

On the left side of the U, we have the Contracting Path, or Encoder. This section works like a typical convolutional neural network. It uses a series of convolution and pooling layers to downsample the input image, gradually reducing its size to capture the broader context.

On the right side, we have the Expanding Path, or Decoder. This path upsamples the reduced feature maps, expanding them back to the original image size. It restores spatial resolution and refines the localisation, which is essential for accurate segmentation.

What makes U-Net especially powerful are the Skip Connections. These connections send high-resolution feature maps directly from the encoder to the decoder. This is important because downsampling often loses fine details. By using skip connections, U-Net allows the decoder to retain those early, detailed features, helping it to produce more accurate pixel-wise segmentation.

In terms of performance, U-Net is excellent for classifying each pixel in an image and works especially well when the available training data is limited.

However, its main weakness is that it's not designed to separate individual instances of the same object class. So, if the image has three cars, U-Net will label all of them as just 'car,' without telling them apart as 'Car 1,' 'Car 2,' or 'Car 3.'

Now, when it comes to instance segmentation, a leading and widely used algorithm is Mask R-C-N-N. This model builds directly on the Faster R-C-N-N architecture used for object detection, but adds an important enhancement—it not only detects where each object is but also segments each one precisely at the pixel level. In other words, it helps identify what the object is, where it is located, and which exact pixels belong to it.

Mask R-C-N-N extends Faster R-C-N-N by introducing a third output branch that predicts segmentation masks. This runs alongside the original two branches: one for classification and one for bounding box regression. The architecture begins with a backbone convolutional neural network, which extracts rich feature maps from the input image. These features are then processed by the Region Proposal Network, or R-P-N, which



suggests potential bounding boxes for objects in the image.

A critical improvement over previous versions is the introduction of the R-O-I Align layer, short for Region of Interest Align. This layer ensures that the features for each proposed object region are perfectly aligned with the original image, allowing for more accurate pixel-level segmentation. This solves the alignment errors caused by older methods like R-O-I Pooling.

After that, the extracted features are passed into the head branches. The classification branch determines what object has been detected—for example, is it a dog, a bicycle, or a tree? The bounding box regression branch refines the coordinates of the proposed box. The third and final branch is a small fully convolutional network, or F-C-N, which predicts a binary segmentation mask for each detected object, but only within the boundaries of its bounding box.

This means that Mask R-C-N-N doesn't just draw a box around an object—it carefully maps out the object itself, pixel by pixel, allowing for precise instance segmentation even when multiple objects of the same type appear in a single image.

Mask R-C-N-N delivers excellent accuracy when it comes to instance segmentation. It doesn't just place bounding boxes around objects—it also produces highly detailed, pixel-level masks for each distinct object in an image. This is why it's widely regarded as state-of-the-art for segmentation tasks.

However, there is a trade-off. Because Mask R-C-N-N performs multiple tasks at once—classification, bounding box regression, and mask prediction—it requires significantly more computational resources than simpler models like standard object detectors or semantic segmentation networks. So, while the results are highly accurate, the cost is higher in terms of memory and processing time.

While U-Net and Mask R-C-N-N represent the modern deep learning standards for segmentation, it's useful to touch on some additional methods. One such approach is Panoptic Segmentation. It aims to combine the strengths of both semantic and instance segmentation. Here, the model outputs a class label and a mask for every single pixel—but also assigns unique IDs to each instance of 'thing' classes like cars or people. In contrast, 'stuff' classes such as sky, grass, or road are treated in a more generalised way—without distinguishing separate objects.

Before deep learning, several traditional techniques were widely used—and they're still helpful today for simpler or pre-processing tasks. Thresholding segments based on pixel intensity—like separating bright objects from a dark background. Clustering algorithms, like K-means, group pixels using patterns in colour or texture. The Watershed Algorithm uses gradients and contours to separate touching or overlapping objects, which is particularly useful in medical imaging.

Even though deep learning dominates now, these traditional methods remain relevant for specific, well-defined problems—or as part of larger segmentation pipelines.

Image segmentation delivers pixel-level precision, making it essential across a wide range of real-world applications.

In autonomous driving, segmentation is absolutely critical. It helps self-driving cars understand their environment by distinguishing between roads, sidewalks, and buildings. It also enables the car to identify drivable areas, obstacles, and pedestrians at the pixel level, ensuring safer and more reliable navigation.

In medical image analysis, segmentation plays a vital role in diagnosing and planning treatment. It allows for precise delineation of tumours, lesions, and other anomalies in X-rays, MRIs, or CT scans. It also supports complex tasks like 3D organ reconstruction and



tracking the progression of diseases.

In robotics, segmentation helps robots gain a more detailed understanding of their surroundings. By knowing the exact shape and boundaries of objects, robots can better grasp and manipulate items. It's also important for navigating complex environments with many overlapping or closely spaced objects.

Augmented reality and virtual try-on applications also rely heavily on segmentation. By accurately separating foreground objects from the background, systems can overlay virtual items like clothing or furniture into real-world scenes, making the experience more immersive and convincing.

Satellite and aerial imagery analysis benefits from segmentation too. It allows for land cover classification—identifying areas like forests, water bodies, agricultural fields, and urban zones—and enables change detection over time, which is valuable for environmental monitoring and urban planning.

And finally, in image editing and photo manipulation, segmentation makes it easy to remove backgrounds automatically and apply effects to specific parts of an image, such as selective colourisation.

These examples highlight how segmentation underpins advanced computer vision solutions that demand detailed, pixel-level visual understanding.

How might you apply image segmentation techniques to solve real-world challenges in your field of study?

Video 6: Computer Vision- Demo

Hello and welcome everyone. In this notebook, we are going to talk about the OpenCV library which we typically use for image processing in Python. We are going to see the features of this library relevant to the kind of task which we do in computer vision.

You might have seen in the lectures about the different techniques such as blurring your image, detecting edges, etcetera and then object detection and further you would see image segmentation. So, let us start to explore how this thing works within Python. So, for that we are going to do some necessary imports.

So, if you see here we are going to import the OpenCV as cv2 for image input output and processing and then we also use a numpy which is an array library. You would see that OpenCV images are nothing but just simple plain numpy arrays under the hood. Further, when we are working in the colab or a Jupyter environment, we import this cv2 underscore im show which is going to help us to render the image inline in the notebook output.

Now, we go by reading the file by providing the path using the cv2 dot im read, remember to provide the correct path over here and this would actually read the file into a numpy array and by default it returns a 3 channel RGB image with the values between 0 to 255. If you need a greyscale, you can pass cv2 dot im read greyscale over here and it would read this image in the greyscale. Now, typically you would see that the coloured image would have a height, a width and a 3-channel corresponding to RGB values.

For a greyscale image, you will not have the channel axis, you can actually try with different combinations, but because I know my image is a coloured image, so I would go as it is. Now, once we have loaded that, we are going to do a check if the image is loaded because if the path is wrong, the file does not exist or maybe sometimes the format cannot be decoded. In that case, OpenCV would return none.



So, we guard against that to avoid any downstream problems. So, we are going to see if the image is really captured, if not, then it would return none. Print you that image and it would exit, that is it is going to stop the cell.

Once we have loaded the image, we can successfully actually go by displaying the image in the colab environment and we see that this is my image. So, I have successfully loaded the image into colab and something which I can now work about. Now, the next thing what we would want to do is, we would want to start with an application of a Gaussian low pass filter.

And why do we do this? You might have seen that in the slides that this helps us to reduce the noise and soften the fine texture. And for that, we use a kernel size, you see here, this is the kernel size. We are using a 15 cross 15, you want to change it, you can change and try, but please keep it an odd number.

Further, this is the sigma x value which is asking OpenCV to infer the sigma from the kernel size. Now, there is a mathematics behind that which we do not have to go into it, but the intuition is larger the kernel, stronger the smoothing. So, you can change this kernel size, and you can see the different effects of that on the output image.

So, I see that clearly you can see that there is a denoising effect, there are softer edges, texture is less. So, this is going to show you the blurred image, not the original image. Now, the next kind of processing technique is to use CANN to produce a binary edge map, a 0 or 255 per pixel.

And this is usually done using a 4-stage pipeline, where you smoothen to suppress the noise, you use the gradient via Sobel filter, you then use a non-maximum suppression and then there is a thresholding. This allows to keep the weak edges only if they connect to the strong ones. So, you see that this CANN would expect a single channel 8-bit image and then it would convert that to greyscale.

So, now if you see here this is basically the range, you are passing this image over here and this is the threshold, this is the low threshold which would include the weak edges, and this is the high threshold which seeds the strong edges. Now, higher thresholds would lead to fewer more cleaner edges, lower thresholds would lead you to more edges and possibly more noise also. So, that is how you pass that threshold and then you can see the edges captured from that image.

So, you see that the blur would reduce the noise. CANN finds the thin connected edge curves using double thresholds. Well, let us try to see what else we can offer.

The next thing is image enhancement. So, per pixel or per RGB channel, we can actually use certain controls for example, alpha which scales the difference from midtones that is it controls the contrast, beta is the offset. Which controls the shift intensities that is the brightness.

So, with the numbers which we came up with 1.5, it would stretch the histogram, the darker pixel would get even darker, the brights would get brighter. And when we say beta equals to 30, we are trying to say that push everything brighter by 30 levels and that is how we convert that to that scale which we are going to provide as parameters over here. This is going to change that perceptual brightness and that is what you then try to see that how the picture is bit more enhanced.

Further, if you want to scale or resize your image, there are three interpolation methods you might have seen that in the slides. We use each of these three and OpenCV offers you the choices of using the nearest, the bilinear and the bicubic. As you know for the nearest, it would pick the closest source pixel and by the looks it would be more crisp, but the looks would be more pixelated, it is the fastest and it uses masks or labels.



In the bilinear format, it would take a weighted average of a 2 cross 2 neighbourhood. So, that by the look it would look more smoother than the previous one, the nearest one and it would be moderately blur also. Bicubic takes the cubic weights over a 4 cross 4 neighbourhood.

By the looks, it would be smooth with better edge, better gradient quality, but it is slower than the other two. So, what are we doing is we are downscaling to 200. So, forcing to 200 cross 200, this will distort the non-square images.

So, to preserve the aspect, we can apply further processing, but for us to keep it simple, these are the three types of scaling which you would see. So, this is how the nearest would look like, this is how the bilinear would look like and this is how the bicubic would look like. So, the three interpolation strategies for resizing, you can see in terms of their implementation, they are just they differ by this interpolation method, whichever interpolation method suits you, you can actually try that.

So, these are some basic image processing techniques. Let us move further into the object detection algorithms. Now, we have seen the first of these algorithm called as the single-shot multibox detector.

And what are we trying to do here is we are trying to pull a pre-trained MobileNet SSD detector in the CAFE format. This model is the classic VOC 20 class SSD. So, we load, we kind of download that model.

And then once we are done with the downloading of that model, we can then read that model by passing on this method and the model. Now, once we have done that, we are going to load our image in the same manner cv2.imread and we get the height and width of the image. The next step is we are going to convert the image to a 4-dimension tensor and apply the exact normalisation as the SSD model expects.

And this will map the pixel range 0 to 255 to roughly minus 1 to 1. You see that the size is 300, 300. This is the SSD input resolution for this model. And we are going to then set this input to forward which keeps the RGB for inference.

And once you do that, it would process the image to a 4-dimension tensor as expected for the SSD model. So, you would see that all the pixels would be normalised to 0 to 1 relative to width and height. Now, what we do is next is we filter out by the confidence.

So, we demormalise the boxes to pixels using this original width and height. So, that is what we are trying to do over here is. So, what did we do? We downloaded the model.

You do not need to go into the details of this, right, but it is we are trying to draw the box. So, we are trying to download the model, we display the image, we pass that through the network so that I can get the detections, the object detections and then we draw the bounding boxes around that. That is going to basically draw the bounding boxes and along with that the confidence score also what you see because we filtered by the confidence for example, we might have kept as 0.5. So, anything above than that would be displayed.

You would see that everything above 0.5 is displayed. So, that is where the model feels more confident about the object. Further, there is another algorithm called as the faster R-CNN.

Here also we go by downloading a tensorflow object detection API for faster R-CNN trained on the COCO type of data set. If we download all the supporting files, I will jump to the implementation. This is how in a similar manner we are going to read, but from tensorflow.



It will parse the frozen tensorflow graph and its OpenCV flavoured the text graph. So, architecturally you know that faster R-CNN is a two-stage detector. So, a region proposal network will propose the candidate boxes and then a second head would classify each proposal.

So, it could be more accurate, but slower than this SSD or the mobile net architecture we have seen above. So, now we read the labels the COCO class names into a python list. We will use this for labels like person, car etcetera and we similarly read the image the same way we had done above.

So, that we can convert the models normalised box coordinates back into the pixel coordinates when we are trying to draw rectangles. Again we pass that through the same process where we create a 4-dimension block from the image. So, faster R-CNN supports variable input size.

So, passing the original would preserve the detail and then the net dot forward would return the detection in the same SSD style layout with normalised box coordinates. All what we need to do is we need to just process the detections and using the same technique where we say that more than 0.5 wherever I feel the confidence is more than that I am going to process that and draw the bounding box around that object. And this is what you would see a very similar sort of output in the same fashion as we saw earlier.

So, that is what actually concludes this image processing techniques. So, the big picture is classification is one label per image, detection is boxes plus classes per object and OpenCV images are RGB images you can use the cv2 underscore I am showing notebooks to present that. For canny you can convert to greyscale, and we saw certain enhancements and resizing techniques then detection techniques using you're the two types of SSD and the faster R-CNN algorithm.

Thank you everyone.